



Multi-agent Systems with Mistral AI, Milvus and Llama-agents

Stephen Batifol | Zilliz

Unstructured Data Meetup, Sept 5th



Speaker



Stephen Batifol

Developer Advocate, Zilliz/ Milvus

✉ stephen.batifol@zilliz.com

[in linkedin.com/in/stephen-batifol/](https://www.linkedin.com/in/stephen-batifol/)

✂ @stephenbt1





LLM
& DATA

Easy Setup

Pip-install to start coding in a notebook within seconds.



Reusable Code

Write once, and deploy with one line of code into the production environment



Integration

Plug into OpenAI, Langchain, LlmIndex, and many more



Feature-rich

Dense & sparse embeddings, filtering, reranking and beyond



Milvus is an open-source vector database for **GenAI** projects. `pip install` on your laptop, plug into popular AI dev tools, and push to production with a single line of code.



28K+

GitHub
Stars



2,600

Forks



25M+

Downloads



250+

Contributors

Seamless integration with all popular AI toolkits



VOYAGE AI



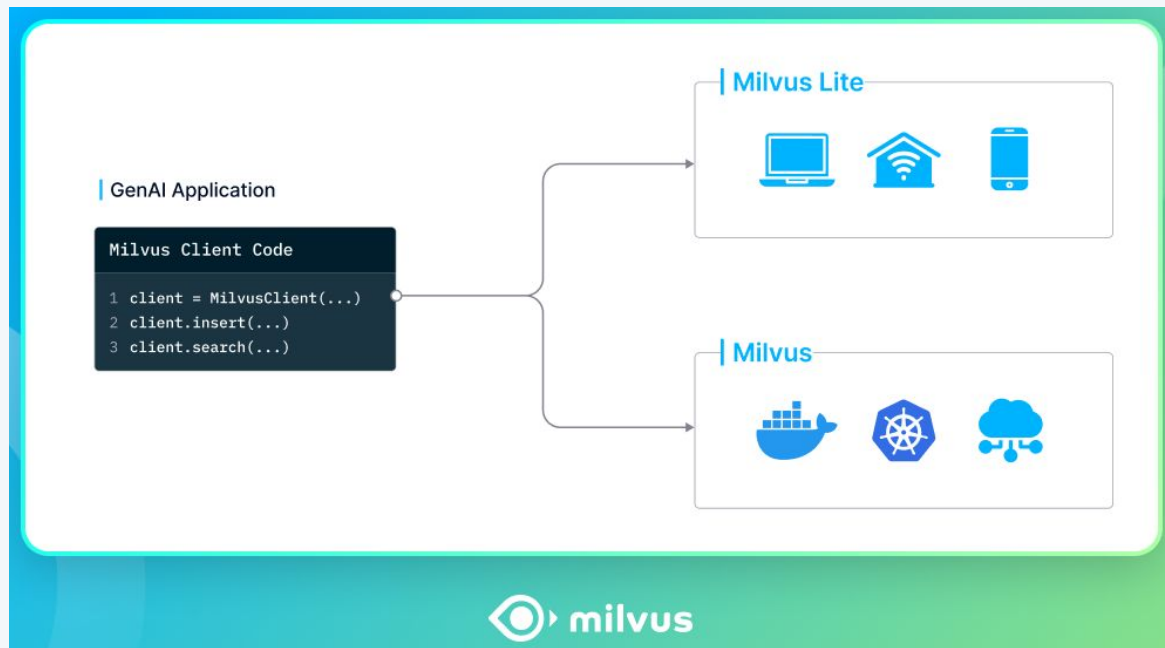
01

Key Components



Milvus Lite

`pip install pymilvus`





llama-index

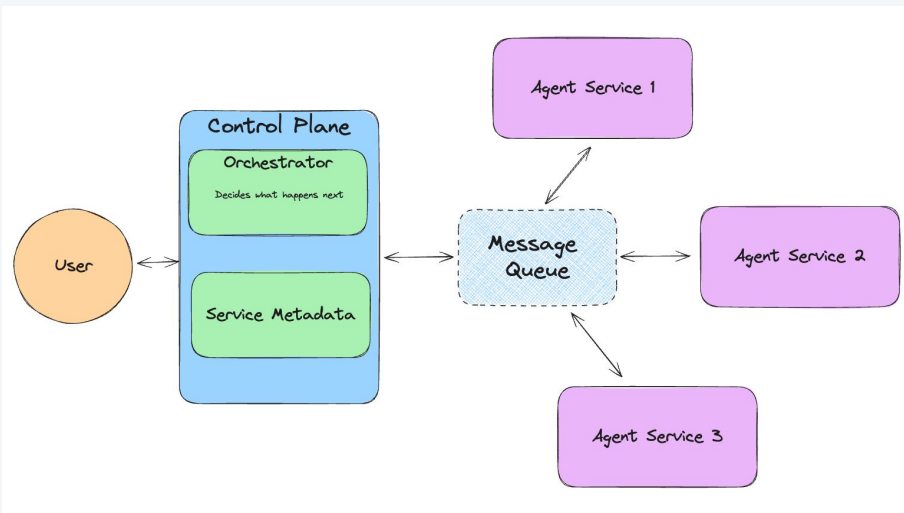
- Framework for building LLM Applications
- Focus on retrieving data and integrating with LLMs
- Integrations with most AI popular tools





llama-agents 🤖 by llama-index

- Build Stateful apps with LLMs and Multi-Agents workflow
- Cycles and Branching
- Human-in-the-Loop
- Persistence



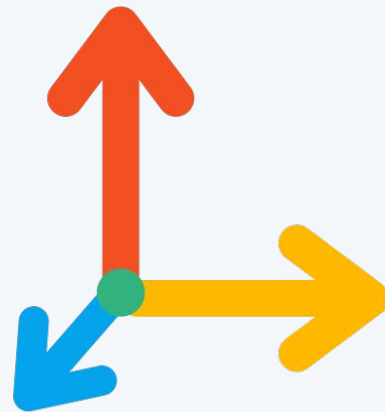
Mistral AI

- **Mistral Embed**
 - Embedding Model focused on Retrieval, very useful for RAG
 - English only
- **Mistral Nemo**
 - 12B model with 128k context length
 - Strong Function Calling and Retrieval for its size
 - Run Locally
- **Mistral Large 2**
 - 123 Billions parameters with 128K context length
 - Very strong Function Calling and Retrieval skills



Tavily

- Web search API for up-to-date information
- Tavily Search API is a search engine optimized for LLMs
- Expands agent's knowledge base



RAG

(Retrieval Augmented Generation)

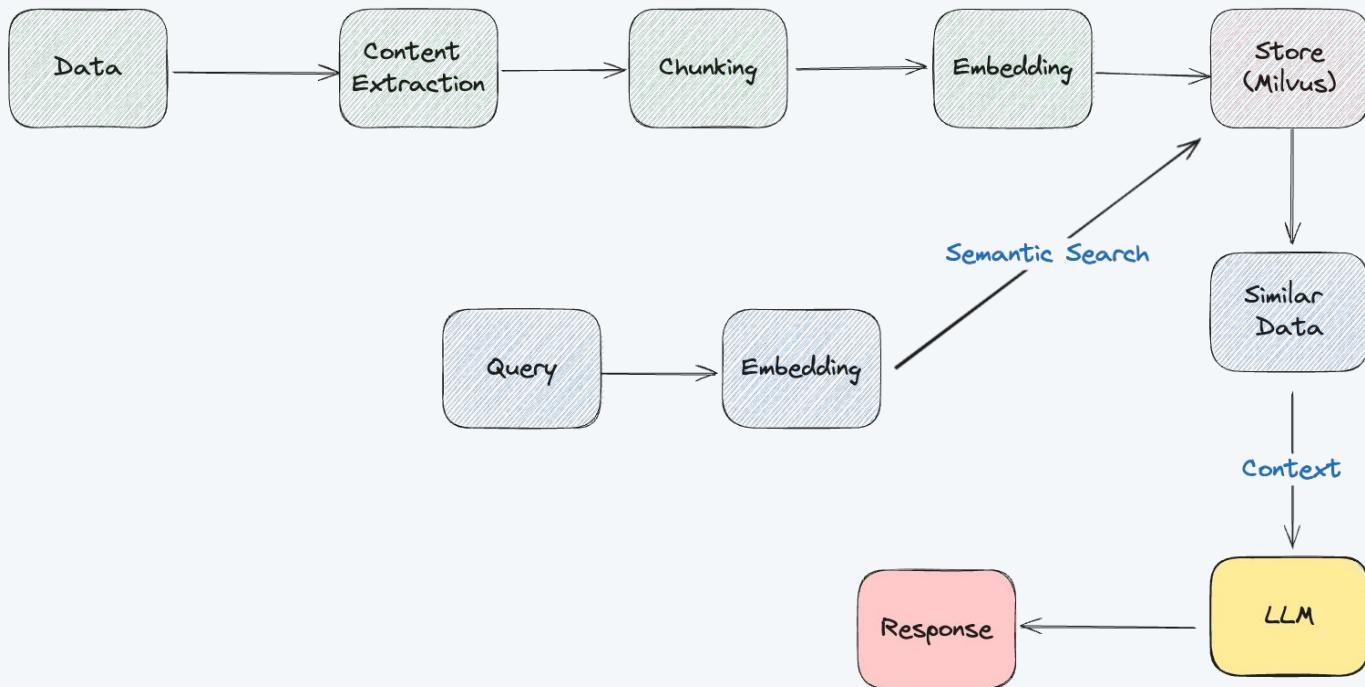


Basic Idea

Use RAG to **force** the **LLM** to work with your data
by injecting it via a vector database like **Milvus**

Basic RAG Architecture

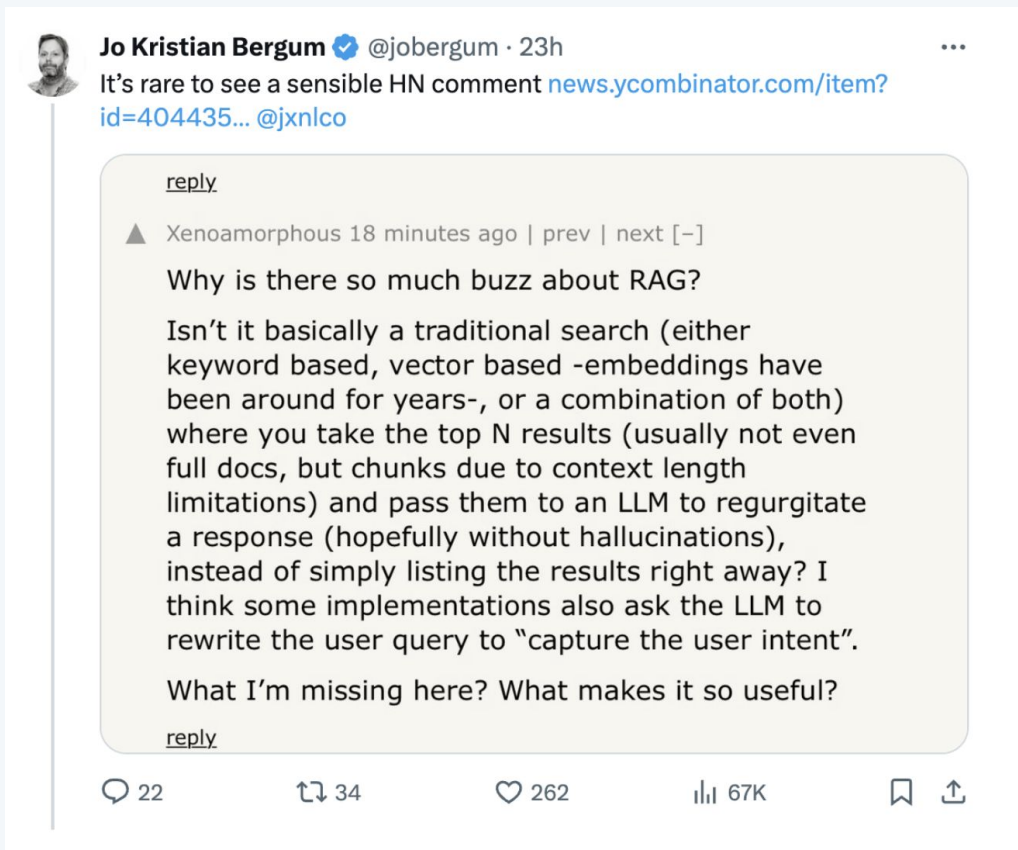
RAG Architecture



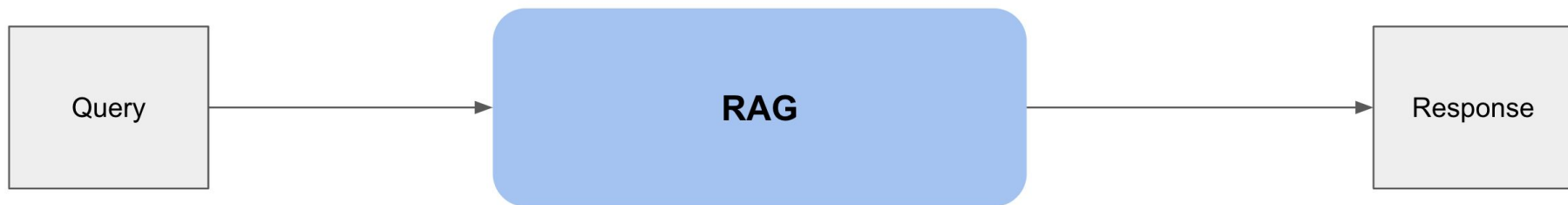
5 lines starter

```
1 # load and parse
2 documents = SimpleDirectoryReader("data").load_data()
3
4 # embed and index
5 index = VectorStoreIndex.from_documents(documents)
6
7 # generate query engine
8 query_engine = index.as_query_engine()
9
10 # run queries
11 response = query_engine.query("What did the author do growing
12 up?")
```

Naive RAG is limited



Naive RAG Pipeline



- ⚠ Single-shot
- ⚠ No query understanding/planning
- ⚠ No tool use
- ⚠ No reflection, error correction
- ⚠ No memory (stateless)

Naive RAG failure mode

Summarization

Naive RAG failure mode

Implicit data

Naive RAG failure mode

Multi-part questions

RAG is necessary but not sufficient

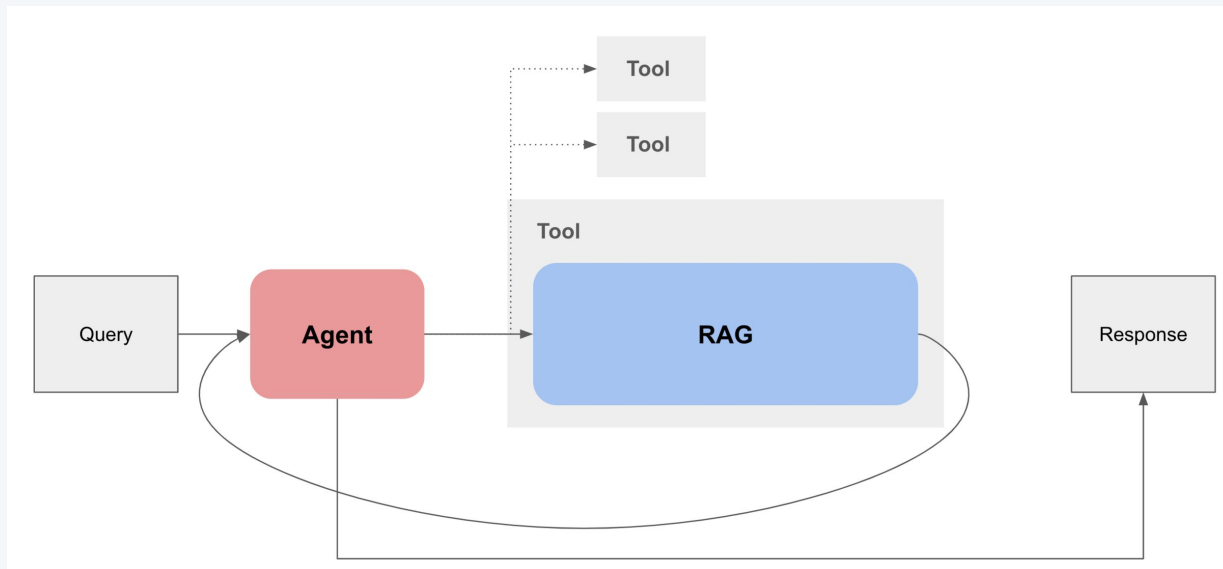


01

Agentic RAG



Agentic RAG



- ✓ Multi-turn
- ✓ Query / task planning layer
- ✓ Tool interface for external environment
- ✓ Reflection
- ✓ Memory for personalization

Agentic RAG

- Self-Reflection 
 - Check the internet to verify
- Query Routing
 - Whether to check our RAG system
- Query Routing with Subquery
 - Whether to check our RAG system for different queries
- Conversation Memory
- Tool Use

Demo!



Thank you



milvus.io



github.com/milvus-io/



[@milvusio](https://twitter.com/milvusio)



[@stephenbtl](https://twitter.com/stephenbtl)



[in/stephen-batifold](https://www.linkedin.com/company/stephen-batifold/)



Milvus Architecture

